

Implementing and Querying Databases with SQL

IEM 4723 Information Systems Design

Dr. Paritosh Ramanan

School of Industrial Engineering and Management
Oklahoma State University

Fall 2026

- 1 Using SQL to Implement a Relational Design
- 2 Simple SQL Retrieval

Using SQL to Implement a Relational Design

The Database Structure Hierarchy

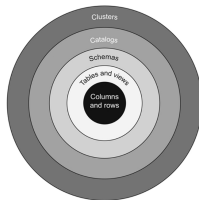


Fig. 11.1 The SQL:2011 database structure hierarchy

- Tables and views that make up a single logical database are collected into a schema.
- Multiple schemas are grouped into catalogs, which can then be grouped into clusters.
- A catalog usually holds information about every schema handled by one DBMS. Catalog creation is implementation dependent, and is not part of the SQL standard itself.
- Clusters, catalogs, and schemas are not required elements of a database environment.

Schemas

- SQL treats a schema simply as a container for tables, views, and other structural elements, and it is up to the database designer to create a meaningful grouping.
- A schema is not required to create tables and views, but is useful when multiple databases share the same DBMS software.

```
CREATE SCHEMA schema_name;  
CREATE SCHEMA schema_name AUTHORIZATION owner_user_ID;
```

- The current schema can be set with `SET SCHEMA schema_name`, or a table can be created directly inside a schema with `CREATE TABLE schema_name.table_name`.

Tables

- There are three categories of tables. Permanent base tables have their contents stored in the database until explicitly deleted. Global temporary tables are working storage destroyed at the end of an SQL session. Local temporary tables are similar, but visible only to the program module that created them.
- Most tables in an RDBMS are permanent base tables, created with `CREATE TABLE`.

```
CREATE TABLE table_name (  
    column1_name column1_data_type column1_constraints,  
    ...  
    table_constraints  
);
```

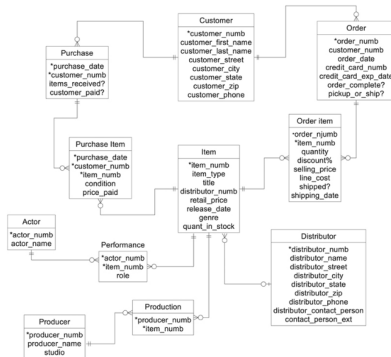
- Table constraints include the declaration of primary and foreign keys, while column constraints include whether a value in that column is mandatory. A default value can be given for a column, such as `order_date date DEFAULT CURRENT_DATE`, and a `NOT NULL` constraint can be added the same way.

Primary and Foreign Keys

- A primary key can be specified in two ways, either by adding a PRIMARY KEY clause to the CREATE TABLE statement, or by adding the keywords PRIMARY KEY directly to the declaration of the column, or columns, that make it up.
- A foreign key is specified with a FOREIGN KEY clause.

```
FOREIGN KEY foreign_key_name (foreign_key_columns)
    REFERENCES primary_key_table (primary_key_columns)
    ON UPDATE update_option
    ON DELETE delete_option
```

The Antique Opticals ER Diagram



- This is the same complete ER diagram for the Antique Opticals database covered earlier in the course. The next few slides show how this design becomes a set of CREATE TABLE statements.

Initial CREATE TABLE Statements

```

CREATE TABLE customer
(customer_num int,
customer_first_name varchar (15),
customer_last_name varchar (15),
customer_street varchar (30),
customer_city varchar (15),
customer_state char (2),
customer_zip char (10),
customer_phone char (12));

CREATE TABLE distributor
(distributor_num int,
distributor_name varchar (15),
distributor_street varchar (30),
distributor_city varchar (15),
distributor_state char (2),
distributor_zip char (10),
distributor_phone char (12),
distributor_contact_person varchar (30),
contact_person_ext char (5));

CREATE TABLE item
(item_num int,
item_type varchar (15),
title varchar (60),
distributor_num int,
retail_price numeric (6,2),
release_date date,
genre varchar (20),
quant_in_stock int);

CREATE TABLE order
(order_num int,
customer_num int,
order_date date,
credit_card_num char (16),
credit_card_exp_date char (5),
order_complete boolean,
pickup_or_ship char (1));

CREATE TABLE order_line
(order_num int,
item_num int,
quantity int,
discount_percent int,
selling_price numeric (6,2),
line_cost numeric (7,2),
shipped boolean,
shipping_date date);

CREATE TABLE purchase
(purchase_date date,
customer_num int,
items_received boolean,
customer_paid boolean);

CREATE TABLE purchase_item
(purchase_date date,
customer_num int,
item_num int,
condition char (15),
price_paid numeric (6,2));

CREATE TABLE actor
(actor_num int,
actor_name varchar (60));

CREATE TABLE performance
(actor_num int,
item_num int,
role varchar (60));

CREATE TABLE producer
(producer_name varchar (60),
studio varchar (40));

CREATE TABLE production
(producer_name varchar (60),
item_num int);

```

Fig. 11.2 Initial CREATE TABLE statements for the Antique Opticals database

- A first pass at the CREATE TABLE statements for the Antique Opticals database, with columns and data types but no keys declared yet.

Adding Primary Keys

```

CREATE TABLE customer
(customer_num int PRIMARY KEY,
customer_first_name varchar (15),
customer_last_name varchar (15),
customer_street varchar (30),
customer_city varchar (15),
customer_state char (2),
customer_zip char (10),
customer_phone char (12));

CREATE TABLE distributor
(distributor_num int PRIMARY KEY,
distributor_name varchar (15),
distributor_street varchar (30),
distributor_city varchar (15),
distributor_state char (2),
distributor_zip char (10),
distributor_phone char (12),
distributor_contact_person varchar (30),
contact_person_ext char (5));

CREATE TABLE item
(item_num int CONSTRAINT item_pk PRIMARY KEY,
item_type varchar (15),
title varchar (60),
distributor_num int,
retail_price numeric (6,2),
release_date date,
genre varchar (20),
quant_in_stock int);

CREATE TABLE order
(order_num int,
customer_num int,
order_date date,
credit_card_num char (16),
credit_card_exp_date char (5),
order_complete boolean,
pickup_or_ship char (1)
PRIMARY KEY (order_num));

CREATE TABLE order_line
(order_num int,
item_num int,
quantity int,
discount_percent int,
selling_price numeric (6,2),
item_cost numeric (7,2),
shipped boolean,
shipping_date date
PRIMARY KEY (order_num, item_num));

CREATE TABLE purchase
(purchase_date date,
customer_num int,
items_received boolean,
customer_paid boolean
PRIMARY KEY (purchase_date, customer_num));

CREATE TABLE purchase_item
(purchase_date date,
customer_num int,
item_num int,
quantity int,
price_paid numeric (6,2)
PRIMARY KEY (purchase_date, customer_num, item_num));

CREATE TABLE actor
(actor_num int PRIMARY KEY,
actor_name varchar (60));

CREATE TABLE performance
(actor_num int,
item_num int,
role varchar (60)
PRIMARY KEY (actor_num, item_num));

CREATE TABLE producer
(producer_name varchar (60) CONSTRAINT producer_pk PRIMARY KEY,
studio varchar (40));

CREATE TABLE production
(producer_name varchar (60),
item_num int
PRIMARY KEY (producer_name, item_num));

```

Fig. 11.3 CREATE TABLE statements including primary key declarations

- The same statements, now with each table's primary key declared, either inline on a single column or added at the end of the statement for tables with a concatenated key.

Adding Foreign Keys

```

CREATE TABLE customer
(customer_num int PRIMARY KEY,
customer_first_name varchar(15),
customer_last_name varchar(15),
customer_street varchar(30),
customer_city varchar(15),
customer_state char(2),
customer_zip char(10),
customer_phone char(12));

CREATE TABLE distributor
(distributor_num int PRIMARY KEY,
distributor_name varchar(15),
distributor_street varchar(30),
distributor_city varchar(15),
distributor_state char(2),
distributor_zip char(10),
distributor_phone char(12),
distributor_contact_person varchar(30),
contact_person_ext char(5));

CREATE TABLE item
(item_num int CONSTRAINT item_pk PRIMARY KEY,
item_type varchar(15),
title varchar(60),
distributor_num int,
retail_price numeric(6,2),
release_date date,
genre varchar(20),
quant_in_stock int);

CREATE TABLE order
(order_num int,
customer_num int,
order_date date,
credit_card_num char(16),
credit_card_exp_date char(5),
order_complete boolean,
pickup_or_ship char(1)
PRIMARY KEY (order_num)
FOREIGN KEY order2customer (customer_num)
REFERENCES customer
ON UPDATE CASCADE
ON DELETE RESTRICT);

CREATE TABLE order_line
(order_num int,
item_num int,
quantity int,
discount_percent int,
selling_price numeric(6,2),
shipping boolean,
shipping_date date
PRIMARY KEY (order_num, item_num)
FOREIGN KEY order_line2item (item_num)
REFERENCES item
ON UPDATE CASCADE
ON DELETE RESTRICT
FOREIGN KEY order_line2order (order_num)
REFERENCES order
ON UPDATE CASCADE
ON DELETE CASCADE);

CREATE TABLE purchase
(purchase_date date,
customer_num int,
item_num int,
items_received boolean,
customer_paid boolean
PRIMARY KEY (purchase_date, customer_num)
FOREIGN KEY purchase2customer (customer_num)
REFERENCES customer
ON UPDATE CASCADE
ON DELETE RESTRICT);

CREATE TABLE purchase_item
(purchase_date date,
customer_num int,
item_num int,
condition char(1),
price_paid numeric(6,2)
PRIMARY KEY (purchase_date, customer_num, item_num)
FOREIGN KEY purchase_item2purchase (purchase_date, customer_num)
ON UPDATE CASCADE
FOREIGN KEY purchase_item2item (item_num)
REFERENCES item
ON UPDATE CASCADE
ON DELETE RESTRICT);

CREATE TABLE actor
(actor_num int PRIMARY KEY,
actor_name varchar(50));

CREATE TABLE performance
(actor_num int,
item_num int,
role varchar(50)
PRIMARY KEY (actor_num, item_num)
FOREIGN KEY performance2actor (actor_num)
REFERENCES actor
ON UPDATE CASCADE
FOREIGN KEY performance2item (item_num)
REFERENCES item
ON UPDATE CASCADE
ON DELETE CASCADE);

CREATE TABLE producer
(producer_name varchar(50) CONSTRAINT producer_pk PRIMARY KEY,
studio varchar(40));

CREATE TABLE production
(producer_name varchar(50),
item_num int
PRIMARY KEY (producer_name, item_num)
FOREIGN KEY production2producer (producer_name)
REFERENCES producer
ON UPDATE CASCADE
ON DELETE CASCADE
FOREIGN KEY production2item
REFERENCES item
ON UPDATE CASCADE
ON DELETE CASCADE);

```

Fig. 11.4 The complete CREATE TABLE statements for the Antique Opticals database

- The complete CREATE TABLE statements, now including every foreign key and its ON UPDATE/ON DELETE behavior, fully implementing the ER diagram in SQL.

Additional Column Constraints

- The **UNIQUE** keyword ensures that all non-null values in a non-primary-key column are unique, such as `ssn char(11) UNIQUE`. It can also be placed at the end of a **CREATE TABLE** statement as **UNIQUE (column_names)**.
- A **CHECK** clause declares a column-specific constraint, such as verifying that a true/false column is limited to T and F.

`CHECK (UPPER(VALUE) = 'T' OR UPPER(VALUE) = 'F')`

Adding Columns and Table Constraints

- The ALTER TABLE statement can add a column to an existing table at any time. Adding two or more columns at once just separates each clause with a comma.

```
ALTER TABLE producer  
ADD producer_phone char (11);
```

- Table constraints, such as foreign keys, can be added the same way.

```
ALTER TABLE customer  
ADD FOREIGN KEY customer2states (customer_state)  
REFERENCES states (state_name);
```

Changing Column Definitions

- The MODIFY command changes an existing column's definition, such as changing `customer_num` from an integer to a character field.

```
ALTER TABLE customer  
MODIFY customer_num char (4);
```

- The DBMS attempts to convert existing values to the new data type, and the change fails if any value cannot be converted. Most columns can be converted to character, but converting character data to numbers or dates requires that the existing values already represent legal values in the new type.
- If the column is referenced by a foreign key, the foreign key columns must be modified too, in order. Remove the foreign key constraint, change the foreign key column, change the primary key column, then add the foreign key constraint back.

Changing Defaults, Nulls, and Constraints

- MODIFY can also change a column's default value, its null status, or a column constraint, without touching anything else about the column.

```
ALTER TABLE order_line  
MODIFY discount_percent DEFAULT 0;
```

```
ALTER TABLE customer  
MODIFY customer_zip NOT NULL;
```

```
ALTER TABLE item  
MODIFY retail_price  
CHECK (VALUE >= 12.95);
```

Deleting Columns and Constraints

```
ALTER TABLE order_line  
DELETE line_cost;
```

```
ALTER TABLE customer  
DELETE CHECK;
```

```
ALTER TABLE customer  
DELETE PRIMARY KEY;
```

```
ALTER TABLE item  
DELETE FOREIGN KEY item2distributor;
```

- Removing a table's primary key means a new one must be added to modify the table further.

Renaming and Dropping

- Renaming a table places the new name after the RENAME keyword, and renaming a column lists both the old and new names, separated by TO.

```
ALTER TABLE order_line  
RENAME line_item;
```

```
ALTER TABLE item  
RENAME title TO item_title;
```

- DROP TABLE *table_name* deletes a table entirely, and this action is irrevocable, since the DBMS will not ask for confirmation. A table cannot be dropped if another table's foreign key still references it. Other elements, such as views, indexes, and domains, can also be removed with DROP.

Simple SQL Retrieval

Introduction to SELECT

- Retrieval is what databases are all about, and SELECT is the SQL command used to retrieve data. It has syntax for choosing columns, choosing rows, combining tables, grouping data, and performing simple calculations.

```
SELECT column1, column2  
FROM table1, table2  
WHERE predicate;
```

- The WHERE clause is optional, and can contain a wide variety of criteria identifying which rows to retrieve.
- Most SQL command processors are not case sensitive about the SQL statement itself, though DBMSs are case sensitive when matching data values. Many DBMSs use a semicolon to terminate a query, and a single command can span many lines.

Choosing Columns

- SQL lets us specify which columns to see and the order to see them in, using the relational algebra project operation.

```
SELECT * FROM work;
```

- The asterisk is a convenient shorthand for a quick overview of the data, and displays columns in the order they appeared in the table's CREATE TABLE statement.

```
SELECT first_name, last_name, contact_phone  
FROM customer;
```

- Listed columns appear in the result in the order given after SELECT.

Removing Duplicates

```
SELECT DISTINCT customer_num, credit_card_num  
FROM sale;
```

- Without DISTINCT, the result table would have one row for every time a customer used a specific credit card.
- DBMSs usually leave duplicates in the result table, since removing them can be computationally expensive, leaving it up to the user to request unique rows.

Ordering the Result Table

```
SELECT * FROM author  
ORDER BY author_first_last;
```

- Sorting is done with an ORDER BY clause. If more than one column is listed, the first represents the outer sort, and later columns sort within it.

```
SELECT zip_postcode, last_name, first_name  
FROM customer  
ORDER BY zip_postcode DESC, last_name ASC;
```

- Ascending or descending order can be specified per column with ASC or DESC.

Choosing Rows

- Row selection criteria go in the `WHERE` clause, and the logical expression that follows it is called a predicate.
- Six common operators express data relationships, `=`, `<`, `<=`, `>`, `>=`, and `!=` or `<>`.

```
cost > 1.95  
numb_on_hand <= reorder_point
```

- How a literal value is written, and how it compares, depends on the column's data type, whether numerical, alphabetical, or chronological order.

Logical Operators

- AND requires every condition to be met, OR requires at least one condition to be met, and NOT inverts the result of a logical expression.

```
condition_code = 2 AND asking_price > 75  
asking_price > 125 OR asking_price < 50  
NOT (asking_price <= 50)
```

- There is no limit to how many simple expressions can be connected together, such as NOT (asking_price <= 50) AND selling_price < asking_price.

Precedence and Special Operators

- A DBMS follows a default rule of precedence unless told otherwise, evaluating simple expressions first, then logical expressions, left to right when more than one operator of the same type appears. Parentheses can override this order.

```
asking_price < 50 OR (condition_code = 2 AND  
    selling_price > asking_price)
```

- Special operators make writing logical criteria easier. BETWEEN checks a range, LIKE allows character-string pattern matching with wildcards, IN checks membership in a list, and IS NULL checks for a missing value.

```
sale_date BETWEEN '01-Aug-2017' AND '30-Sep-2017'  
selling_price IN (25, 50, 60)  
sale_date IS NULL
```

Row Selection Examples

- Using a primary key expression retrieves exactly one row.

```
SELECT first_name, last_name, contact_phone  
FROM customer  
WHERE customer_num = 6;
```

- A concatenated primary key uses a complex predicate, with each key column in its own simple expression.

```
SELECT child_birth_date  
FROM dependents  
WHERE employee_number = '111333' AND child_name = '  
    John';
```

Complex Predicates

```
SELECT sale_id, sale_total_amt  
FROM sale  
WHERE sale_date < '1-Aug-2017' AND sale_total_amt < 100;
```

- Parentheses make a complex predicate easier to read, and can be used even when they do not change the evaluation order.

```
SELECT sale_id, sale_date, sale_total_amt  
FROM sale  
WHERE sale_date BETWEEN '1-Aug-2017' AND '31-Aug-2017'  
ORDER BY sale_date;
```

- NOT BETWEEN excludes the given range instead of selecting it.